Integrantes:

Nicolas de Paula Pacheco – RGM – 47168536
Gabriel de Oliveira Silva – RGM – 47121645
Marco Antonio – 47469145

Exemplos

Link da calculadora: https://java-calculator--gabrielmvn.replit.app

Exemplo 1 - CRIE UMA CALCULADORA EM JAVA

```
import java.util.Scanner;
public class Calculadora {
public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);
boolean continuar = true;
System.out.println("=== CALCULADORA ===");
while (continuar) {
System.out.println("\nEscolha a operação:");
System.out.println("1 - Adição (+)");
System.out.println("2 - Subtração (-)");
System.out.println("3 - Multiplicação (×)");
System.out.println("4 - Divisão (÷)");
System.out.println("5 - Porcentagem (%)");
System.out.println("0 - Sair");
System.out.print("\nOpção: ");
int opcao = scanner.nextInt();
if (opcao == 0) {
System.out.println("Encerrando a calculadora. Até logo!");
break;
}
if (opcao < 1 || opcao > 5) {
System.out.println("Opção inválida. Tente novamente.");
continue;
}
System.out.print("Digite o primeiro número: ");
double num1 = scanner.nextDouble();
System.out.print("Digite o segundo número: ");
double num2 = scanner.nextDouble();
double resultado;
switch (opcao) {
case 1:
resultado = num1 + num2;
System.out.printf("Resultado: %.2f + %.2f = %.2f%n", num1, num2,
resultado);
break;
case 2:
resultado = num1 - num2;
```

```
System.out.printf("Resultado: %.2f - %.2f = %.2f%n", num1, num2,
resultado);
break;
case 3:
resultado = num1 * num2;
System.out.printf("Resultado: %.2f × %.2f = %.2f%n", num1, num2,
resultado);
break;
case 4:
if (num2 == 0) {
System.out.println("Erro: Divisão por zero não é permitida.");
} else {
resultado = num1 / num2;
System.out.printf("Resultado: %.2f ÷ %.2f = %.2f%n", num1, num2,
resultado);
}
break;
case 5:
resultado = (num1 * num2) / 100;
System.out.printf("Resultado: %.2f%% de %.2f = %.2f%n", num2, num1,
resultado);
break;
default:
System.out.println("Opção inválida.");
}
}
scanner.close();
}
}
```

Possíveis problemas

- Entrada inválida: se o usuário digitar letras ou símbolos em vez de números, o programa vai lançar uma exceção (InputMismatchException). Isso não é tratado no código.
- Histórico de cálculos: atualmente, o programa não guarda histórico. Cada resultado é mostrado na tela e depois perdido. Se você quiser ver os cálculos anteriores, precisaria implementar uma estrutura para armazená-los (por exemplo, uma ArrayList<String> que guarda cada operação realizada e imprime no final ou sob demanda).
- Porcentagem confusa: o texto exibido pode gerar dúvida. Exemplo: se o usuário digitar num1 = 200 e num2 = 10, o programa mostra “10% de 200 = 20”. Isso está correto, mas a ordem dos números pode confundir quem espera “200% de 10”. Talvez seja melhor deixar claro na mensagem qual número é a base e qual é a porcentagem.

Exemplo 2 – Crie uma lista de tarefas em JAVA

```
import java.util.ArrayList;
import java.util.Scanner;

public class ListaDeTarefas {
    public static void main(String[] args) {
        ArrayList<String> tarefas = new ArrayList<>();
        Scanner scanner = new Scanner(System.in);
        int opcao;

        do {
            System.out.println("\n--- MENU ---");
            System.out.println("1 - Adicionar tarefa");
            System.out.println("2 - Listar tarefas");
            System.out.println("3 - Remover tarefa");
            System.out.println("0 - Sair");
            System.out.print("Escolha uma opção: ");
            opcao = scanner.nextInt();
            scanner.nextLine(); // limpar buffer

            switch (opcao) {
                case 1:
                    System.out.print("Digite a tarefa: ");
                    String tarefa = scanner.nextLine();
                    tarefas.add(tarefa);
                    System.out.println("Tarefa adicionada!");
                    break;

                case 2:
```

```
            System.out.println("\n--- LISTA DE TAREFAS ---");
            if (tarefas.isEmpty()) {
                System.out.println("Nenhuma tarefa cadastrada.");
            } else {
                for (int i = 0; i < tarefas.size(); i++) {
                    System.out.println((i + 1) + " - " + tarefas.get(i));
                }
            }
            break;

        case 3:
            System.out.print("Digite o número da tarefa para remover: ");
            int indice = scanner.nextInt() - 1;

            if (indice >= 0 && indice < tarefas.size()) {
                tarefas.remove(indice);
                System.out.println("Tarefa removida!");
            } else {
                System.out.println("Número inválido.");
            }
            break;

        case 0:
            System.out.println("Saindo...");
            break;

        default:
            System.out.println("Opção inválida!");
    }
```

```
        } while (opcao != 0);


        scanner.close();
    }
}
```

Erros no código-

As tarefas são armazenadas na memória RAM
Ou seja:

- Se você fechar o programa → perde tudo
- Não salva em arquivo nem banco de dados
- Problema: Não existe persistência de dados
- ArrayList<String> tarefas = new ArrayList<>();

Você salva apenas texto da tarefa
Não existe:

- status (pendente/concluído)
- checkbox
- controle de progresso
- Problema: Não dá pra marcar tarefa como concluída
- ArrayList<String> tarefas

Possível erro ao remover tarefa

Trecho:

```
int indice = scanner.nextInt() - 1;
```

Problema:

- Se o usuário digitar letra → erro (InputMismatchException)
- Se digitar número inválido → já tratou parcialmente ✔

Melhorar: Validar entrada do usuário

Interface muito simples (console)

Funciona
Mas:

- Não é visual
- Não é intuitivo como app real

Exemplo 3 – API Flask

Possiveis Ajustes

1. **Indentação**: no trecho da função `home()`, o `return` precisa estar indentado corretamente.

```
@app.route('/')

def home():

    return "API Python funcionando"
```

Se não estiver com esse recuo, o Python vai lançar erro de sintaxe.

2. **Execução recomendada**: em projetos Flask, é boa prática usar o bloco `if __name__ == "__main__":` para garantir que o servidor só rode quando o arquivo for executado diretamente:

```
if __name__ == "__main__":

    app.run(debug=True)
```

O parâmetro `debug=True` ajuda durante o desenvolvimento, mostrando erros detalhados e recarregando automaticamente quando você altera o código.

3. **Histórico de requisições**: diferente da calculadora em Java, aqui você não terá um "histórico" de chamadas. Cada requisição ao endpoint / retorna apenas a mensagem fixa. Se quiser registrar acessos, pode usar **logs** ou armazenar em uma lista/banco de dados.

```
from flask import Flask



app = Flask(__name__)
```

```
@app.route('/')
def home():
    return "API Python funcionando"


if __name__ == "__main__":
    app.run(debug=True)
```

Possiveis erros :

1-from flask import Flask

Correto. Importa a classe Flask da biblioteca.

Possível erro: se o Flask não estiver instalado (pip install flask), o programa não roda.

2-@app.route('/')

def home():

return "API Python funcionando"

Erro de indentação: o return precisa estar indentado com 4 espaços dentro da função. Assim:

3-@app.route('/')

def home():

    return "API Python funcionando"

Se não corrigir, o Python lança `IndentationError`.

4-app.run()

Funciona, inicia o servidor Flask na porta padrão 5000.

Pontos de atenção:

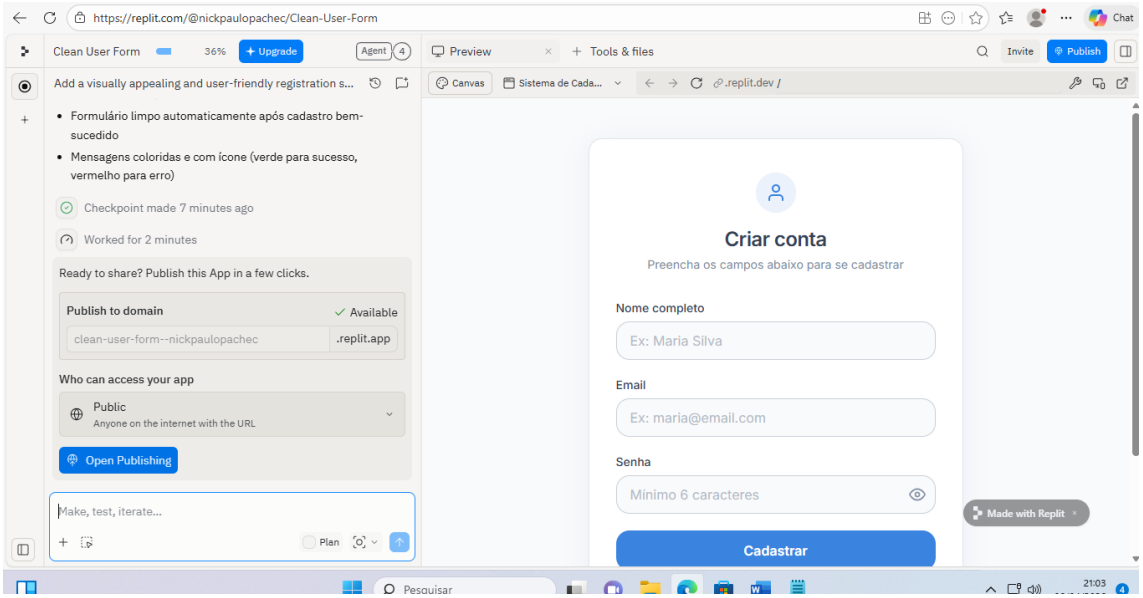
- Sem o bloco if __name__ == "__main__":, o servidor pode iniciar em situações indesejadas (por exemplo, se o arquivo for importado como módulo).
- Não está em modo debug. Para desenvolvimento, é útil usar:

```
if __name__ == "__main__":

    app.run(debug=True)
```

EXERCÍCIO 1

• Criar um sistema de cadastro em Python usando IA
• Gerar código com IA e explicar cada linha
• Encontrar erros no código gerado



Prompt:

Você é um desenvolvedor experiente em Python e design de interfaces simples e intuitivas.

Crie um sistema de cadastro em Python com foco em uma experiência leve, limpa e agradável para o usuário, evitando poluição visual e cansaço nos olhos.

Requisitos do sistema:

Interface:

Utilize uma interface gráfica (Tkinter ou outra biblioteca simples).

Use cores suaves e claras (tons pastéis ou neutros, como branco, cinza claro, azul suave).

Evite cores muito fortes ou chamativas.

Fonte legível e de tamanho confortável.

Espaçamento adequado entre os elementos.

Layout centralizado e organizado.

Funcionalidades:

Cadastro de usuário com os seguintes campos:

Nome

Email

Senha

Botão de "Cadastrar"

Botão de "Limpar campos"

Exibir mensagem de sucesso ou erro de forma amigável.

Validações:

Não permitir campos vazios.

Validar formato básico de email.

Exibir mensagens claras e simples (ex: "Por favor, preencha todos os campos").

Experiência do usuário:

Interface intuitiva, que qualquer iniciante consiga usar sem instruções.

Feedback visual ao clicar nos botões.

Mensagens curtas e amigáveis.

Código:

Código bem organizado e comentado.

Separação de funções (ex: validação, interface, ações dos botões).

Fácil de entender para iniciantes.

Objetivo:

Criar um sistema de cadastro funcional e visualmente confortável, que proporcione uma experiência leve e agradável ao usuário.

Ao final, forneça o código completo funcionando.

Encontrando erros no código e comentando:

// ⚠ BUG 1 — Falta de .trim() no e-mail

// Se o usuário digitar " ana@email.com" (com espaço antes), o Zod

// avalia a string incluindo o espaço e rejeita com "formato inválido",

```
  // mesmo o e-mail sendo correto. Solução: adicionar .trim() antes de .email().
  // Correto seria: z.string().trim().min(1, ...).email(...)
  email: z
    .string()
    .min(1, "Por favor, informe o seu e-mail.")
    .email("O formato do e-mail não é válido."),
  senha: z
    .string()
    .min(6, "A senha deve ter pelo menos 6 caracteres.")
    .max(50, "A senha deve ter no máximo 50 caracteres."),
});
type CadastroFormData = z.infer<typeof cadastroSchema>;
/* ————————————————————————————————————
   Tipos de feedback para o usuário
   ———————————————————————————————————— */
// ⚠ BUG 2 — Variante "erro" nunca é utilizada na prática
// O tipo Feedback permite tipo: "erro", e o JSX até renderiza um ícone vermelho
// para esse caso. Porém, em nenhum lugar do código `setFeedback({ tipo: "erro", ... })`
// é chamado — o onSubmit sempre define tipo: "sucesso".
// Se a chamada à API falhar no futuro, o bloco de erro na tela nunca será exibido.
// Solução: implementar um bloco try/catch no onSubmit e chamar setFeedback com
tipo: "erro"
// quando a requisição falhar.
type Feedback =
  | { tipo: "sucesso"; mensagem: string }
  | { tipo: "erro"; mensagem: string }
  | null;
/* ————————————————————————————————————
   Componente principal da página de Cadastro
   ———————————————————————————————————— */
```

```
export default function CadastroPage() {
  const [feedback, setFeedback] = useState<Feedback>(null);
  const [mostrarSenha, setMostrarSenha] = useState(false);
  const [carregando, setCarregando] = useState(false);
  /* Configuração do formulário com valores padrão */
  const form = useForm<CadastroFormData>({
    resolver: zodResolver(cadastroSchema),
    defaultValues: {
      nome: "",
      email: "",
      senha: "",
    },
  });
  /* — Ação: Cadastrar — */
  function onSubmit(dados: CadastroFormData) {
    setCarregando(true);
    setFeedback(null);
    // ⚠ BUG 3 — setTimeout sem limpeza (vazamento de memória)
    // O ID retornado por setTimeout não é armazenado nem cancelado.
    // Se o usuário navegar para outra página antes dos 900ms terminarem,
    // o callback ainda tentará chamar setCarregando, setFeedback e form.reset()
    // em um componente desmontado, causando erros no React.
    // Solução: guardar o ID em um useRef e cancelar com clearTimeout no cleanup
    // de um useEffect, ou usar uma flag de "montado" (isMounted).
    setTimeout(() => {
      setCarregando(false);
      /* Em um sistema real, aqui você enviaria os dados para o servidor.
         Por ora, simulamos sempre um cadastro bem-sucedido. */
      setFeedback({
```

```
      tipo: "sucesso",
      mensagem: `Bem-vindo(a), ${dados.nome}! Seu cadastro foi realizado com sucesso.`,
    });
    // ⚠ BUG 4 — mostrarSenha não é redefinido após o reset
    // form.reset() limpa os campos do formulário, mas o estado `mostrarSenha`
    // continua true se o usuário tiver clicado em "mostrar senha" antes de enviar.
    // Isso faz com que, após o cadastro, o novo campo de senha apareça como texto
    // visível ao invés de estar oculto por padrão.
    // Solução: chamar setMostrarSenha(false) junto com form.reset().
    form.reset();
  }, 900);
 }
 /* — Ação: Limpar campos — */
 function onLimpar() {
  form.reset();
  // ⚠ BUG 5 — Mesmo problema do BUG 4 aqui
  // Ao limpar o formulário, o campo de senha pode continuar mostrando o texto
  // caso o usuário tenha ativado "mostrar senha" antes de clicar em Limpar.
  // Solução: adicionar setMostrarSenha(false) aqui também.
  setFeedback(null);
 }
```

Exercicio 2

- ***Prompt do jogo:*** Criar um jogo no estilo infinito com IA, onde física dos pulos funcionando, plataformas e pontuação, quero que cada 10 níveis mudem de cor a tela, quero de adicione dificuldade, vidas extra cada 15 níveis, quero que a pessoa possa escolher seu personagem, rank de pontuação

- ***Qual estilo?***
- ***R:*** *Infinite Jump*

- ***Qual plataforma?***
- ***R****: Replit*

- ***Qual pontuação?***
- ***R:*** *por níveis passados****,*** *Top 10 salvo localmente*
- *Medalhas para os 3 primeiros (ouro, prata, bronze)*
- *Aparece o personagem usado e data da jogada*

Link do jogo: *https://infinite-jump-ai--mantonioliracav.replit.app*